

100. Faire clignoter un personnage touché

Touché : le héros clignote une seconde. Une palette qui s'allume et s'éteint, rien de plus.

1. Ce qu'il faut comprendre

Dans presque tous les jeux, un héros touché **clignote** un moment : il est invincible tant qu'il clignote. On pourrait le cacher et le remonter avec `cacher()` et `sprite()` ; il y a plus simple.

`paletteLutins(0, 0, 0, 0, 0)` fait de ses trois nuances visibles... la nuance 0, le blanc du fond : le héros **disparaît**, sans bouger d'un pixel. `paletteLutins(0, 0, 1, 2, 3)` le fait revenir. Alternier les deux toutes les quatre images, c'est le clignotement.

`touche` compte les images d'invincibilité qui restent : 60 au moment du coup, puis un de moins à chaque image. `touche % 8 < 4` est vrai quatre images sur huit : le modulo du chapitre 1 fait le rythme. Quand `touche` retombe à 0, la palette normale revient pour de bon.

La garde `touche == 0` dans le `if` du bouton empêche de relancer le clignotement tant qu'il dure — c'est exactement la règle « invincible pendant une seconde ».

Attention : la palette s'applique à **tous** les lutins qui l'emploient. Pour que seul le héros clignote, ses ennemis prennent l'autre palette (`PALETTE1`).

À toi : fais clignoter l'écran entier à la place, avec `paletteFond`.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <paletteLutins>` → [98.1. La fonction paletteLutins\(\) — les nuances des lutins](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **569 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	57 o	← la console, à l'allumage — la mise en route de la console
main()	104 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
RangerLutins	11 o	← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles	94 o	← démarrage — copie les lettres et les dessins en mémoire vidéo

```

EffacerCarte      14 o ← démarrage – vide les deux cartes de l'écran
LireManette      37 o ← bouton() – lit les huit boutons
Tuiles           144 o ← CopierTuiles – 8 lettres sur 43, et 1 tuile dessinée
DonneesTransfert  10 o ← InstallerTransfert – la petite routine qui copie les lutins
"A: TOUCHE"      9 o ← texte() – un texte gravé

```

Lettres gravées : ACEHOTU:

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

```

sprite()      × 1    23 o sur place, dans main()
texte()       × 1    11 o sur place, dans main() + EcrireTexte (10 o)
paletteLutins() × 2    8 o sur place, dans main()
image()       × 1    6 o sur place, dans main() + AttendreImage (28 o)
bouton()      × 1    5 o sur place, dans main() + LireManette (37 o)

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : un héros au milieu de l'écran. Appuie sur A : il est « touché »
// et clignote pendant une seconde (60 images).
//
// L'astuce : pour le cacher, on ne l'efface pas. On met toutes les nuances
// de sa palette à 0 (le blanc du fond) : il devient invisible, sans bouger.

// Le héros, en signes : . = 0 (transparent), - = 1, # = 3.

#include <Tuile>          // un dessin de 8 × 8 pixels
#include <texte>           // écrit un texte à l'écran
#include <sprite>         // place un lutin de 8 × 8 au pixel près
#include <bouton>         // lit un bouton de la manette
#include <paletteLutins>  // choisit les quatre nuances des lutins

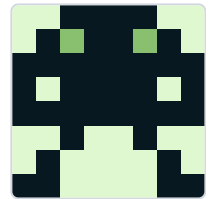
```

2. UN DESSIN — HEROS

```

Tuile HEROS = {
    "..####..",
    ".#-##-#.",
    "#####",
    "#.####.#",
    "#####",
    "..#..#..",
    ".#...#.",
    "##...##",
};

```



à l'écran

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    texte(1, 1, "A: TOUCHE");
    sprite(0, 76, 70, HEROS);              // Lutin 0, au pixel (76, 70) : au milieu.

    uint8_t touche = 0;                    // les images d'invincibilité qui restent (0 = pas touché)

    while (true) {                          // La boucle du jeu :
        image();

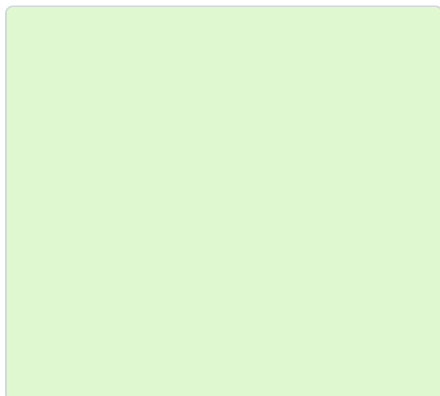
        if (bouton(A) && touche == 0) touche = 60; // A, et pas déjà en train de clignoter :
                                                    // 60 images d'invincibilité (1 seconde).

        if (touche > 0 && touche % 8 < 4) { // Pendant le clignotement, 4 images sur 8 :
                                                    // touche % 8 = le reste par 8 (0 à 7) ;
                                                    // < 4 est vrai pour 0, 1, 2, 3.
            paletteLutins(0, 0, 0, 0, 0); // invisible : toutes ses nuances = 0 (le fond)
        } else {                               // Les 4 autres images, ou quand ce n'est pas touché :
            paletteLutins(0, 0, 1, 2, 3); // visible : la palette normale
        }

        if (touche > 0) touche--;              // Une image de moins à attendre (sans passer sous 0).
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.

A: TOUCHE

A — l'écran a changé

5. Ce qu'on doit voir

Un petit héros au milieu ; A le fait clignoter une seconde, puis il reste visible.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au repos, le héros est visible
- ✓ touché, il apparaît et disparaît (nuances vues : 3, 0)
- ✓ une seconde plus tard, il reste visible

Ensuite : **100.1. Chapitre 16 — La couleur : la Game Boy Color : ce qui vient** — chapitre 16 : La couleur : la Game Boy Color. ·
gameboy3 — écrire une cartouche Game Boy en C++